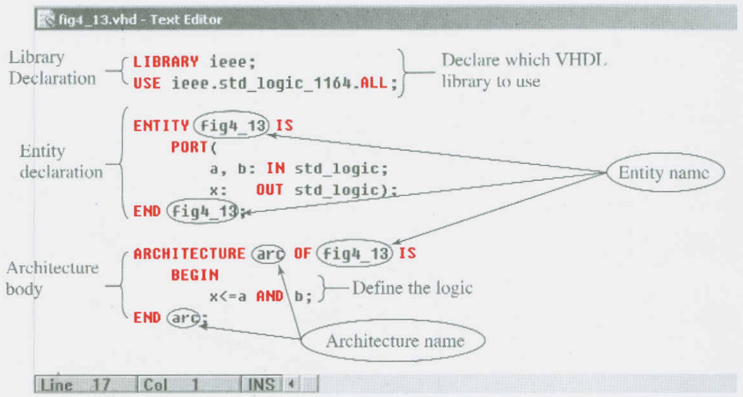
❶

Digital Electronics with VHDL
William Kleitz
VHDL by Example



```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY fig4_13 IS
    PORT(
        a, b: IN std_logic;
        x:   OUT std_logic);
END fig4_13;

ARCHITECTURE arc OF fig4_13 IS
    BEGIN
        x<=a AND b;
END arc;
```

Library Declaration

Declare which VHDL library to use

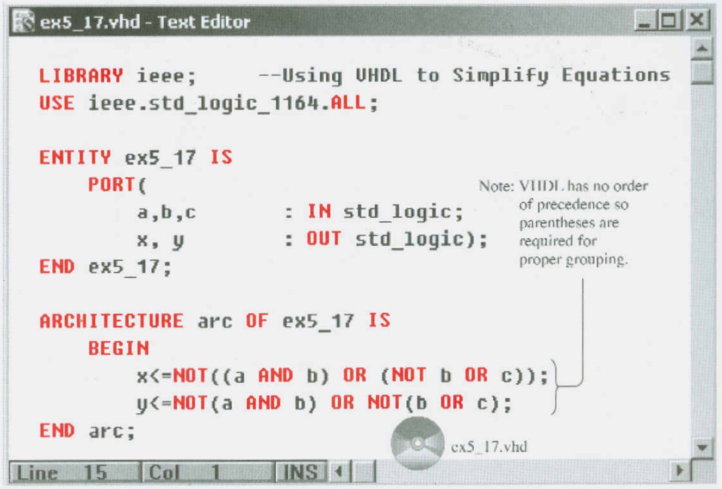
Entity declaration

Entity name

Architecture body

Define the logic

Architecture name

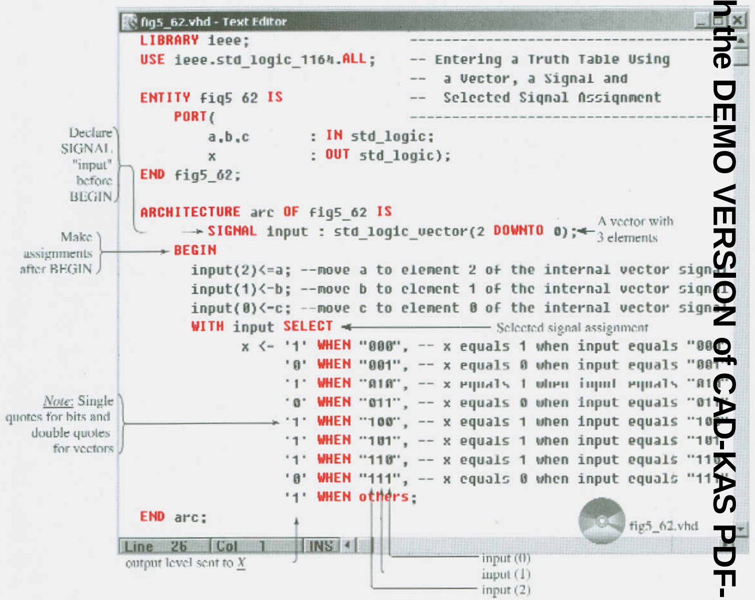


```
LIBRARY ieee;     --Using VHDL to Simplify Equations
USE ieee.std_logic_1164.ALL;

ENTITY ex5_17 IS
    PORT(
        a,b,c       : IN std_logic;
        x, y        : OUT std_logic);
END ex5_17;

ARCHITECTURE arc OF ex5_17 IS
    BEGIN
        x<=NOT((a AND b) OR (NOT b OR c));
        y<=NOT(a AND b) OR NOT(b OR c);
END arc;
```

Note: VHDL has no order of precedence so parentheses are required for proper grouping.

©2004 by Pearson Education, Inc.

❷



```
LIBRARY ieee;                           -------------------------------
USE ieee.std_logic_1164.ALL;            -- Entering a Truth Table Using
                                        --  a Vector, a Signal and
ENTITY fig5 62 IS                       --  Selected Signal Assignment
    PORT(                               -------------------------------
        a,b,c       : IN std_logic;
        x           : OUT std_logic);
END fig5_62;

ARCHITECTURE arc OF fig5_62 IS
    SIGNAL input : std_logic_vector(2 DOWNTO 0);
    BEGIN
     input(2)<=a; --move a to element 2 of the internal vector sign
     input(1)<-b; --move b to element 1 of the internal vector sign
     input(0)<-c; --move c to element 0 of the internal vector sign
     WITH input SELECT
          x <- '1' WHEN "000", -- x equals 1 when input equals "00
               '0' WHEN "001", -- x equals 0 when input equals "00
               '1' WHEN "010", -- x equals 1 when input equals "01
               '0' WHEN "011", -- x equals 0 when input equals "01
               '1' WHEN "100", -- x equals 1 when input equals "10
               '1' WHEN "101", -- x equals 1 when input equals "10
               '1' WHEN "110", -- x equals 1 when input equals "11
               '0' WHEN "111", -- x equals 0 when input equals "11
               '1' WHEN others;
END arc;
```

Declare SIGNAL "input" before BEGIN

Make assignments after BEGIN

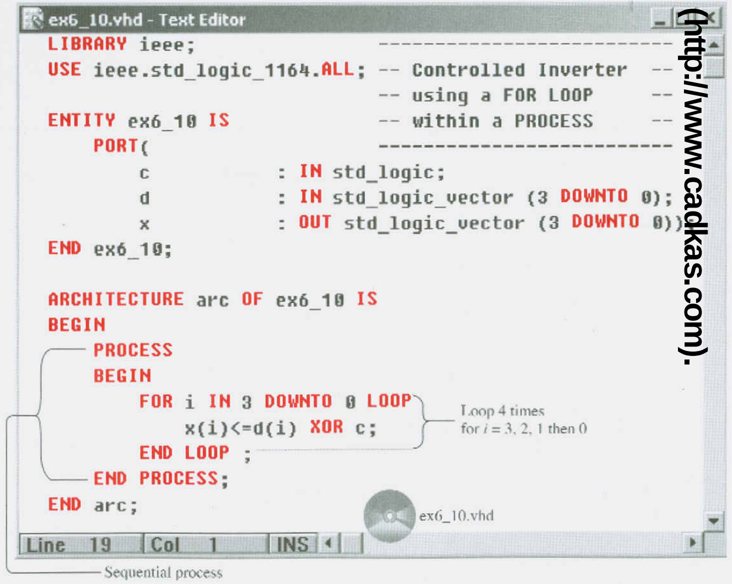
A vector with 3 elements

Selected signal assignment

Note: Single quotes for bits and double quotes for vectors

output level sent to *X*

input (0)
input (1)
input (2)

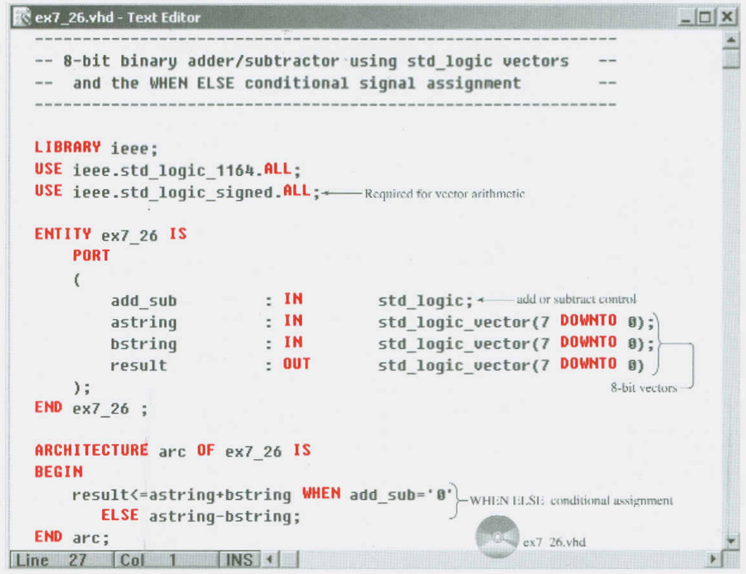


```
LIBRARY ieee;                   ---------------------------
USE ieee.std_logic_1164.ALL;    -- Controlled Inverter   --
                                -- using a FOR LOOP       --
ENTITY ex6_10 IS                -- within a PROCESS       --
    PORT(                       ---------------------------
        c           : IN std_logic;
        d           : IN std_logic_vector (3 DOWNTO 0);
        x           : OUT std_logic_vector (3 DOWNTO 0));
END ex6_10;

ARCHITECTURE arc OF ex6_10 IS
BEGIN
    PROCESS
    BEGIN
        FOR i IN 3 DOWNTO 0 LOOP
            x(i)<=d(i) XOR c;
        END LOOP ;
    END PROCESS;
END arc;
```

Loop 4 times for *i* = 3, 2, 1 then 0

Sequential process

❸



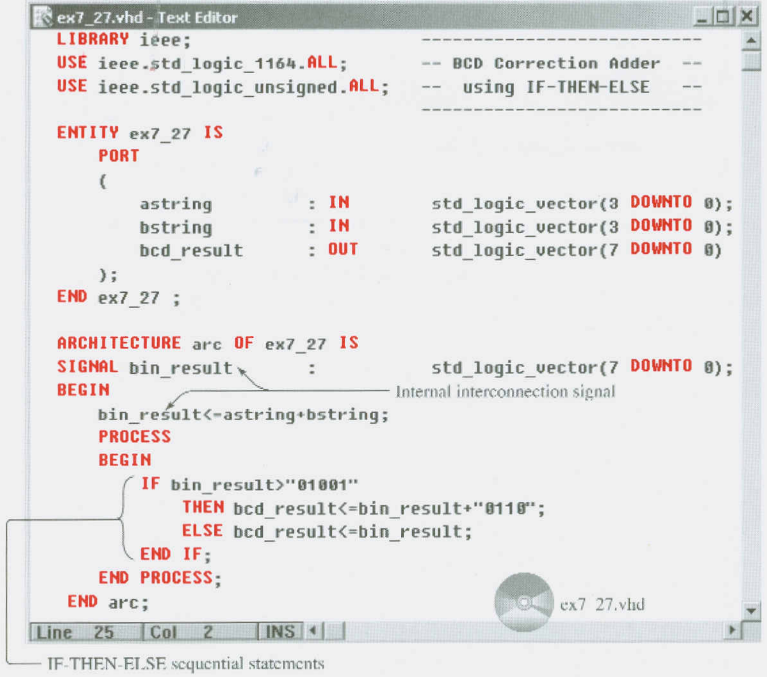
```
---------------------------------------------------------
-- 8-bit binary adder/subtractor using std_logic vectors --
--  and the WHEN ELSE conditional signal assignment      --
---------------------------------------------------------

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.std_logic_signed.ALL;

ENTITY ex7_26 IS
    PORT
    (
        add_sub         : IN    std_logic;
        astring         : IN    std_logic_vector(7 DOWNTO 0);
        bstring         : IN    std_logic_vector(7 DOWNTO 0);
        result          : OUT   std_logic_vector(7 DOWNTO 0)
    );
END ex7_26 ;

ARCHITECTURE arc OF ex7_26 IS
BEGIN
    result<=astring+bstring WHEN add_sub='0'
        ELSE astring-bstring;
END arc;
```

Required for vector arithmetic

add or subtract control

8-bit vectors

WHEN ELSE conditional assignment

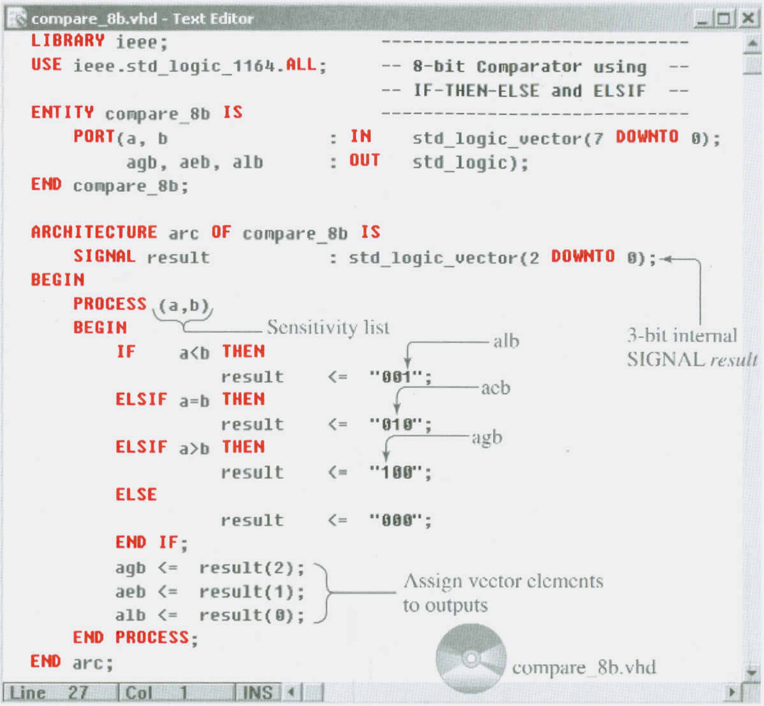


```
LIBRARY ieee;                       ------------------------
USE ieee.std_logic_1164.ALL;        -- BCD Correction Adder --
USE ieee.std_logic_unsigned.ALL;    --  using IF-THEN-ELSE  --
                                    ------------------------
ENTITY ex7_27 IS
    PORT
    (
        astring         : IN    std_logic_vector(3 DOWNTO 0);
        bstring         : IN    std_logic_vector(3 DOWNTO 0);
        bcd_result      : OUT   std_logic_vector(7 DOWNTO 0)
    );
END ex7_27 ;

ARCHITECTURE arc OF ex7_27 IS
SIGNAL bin_result       :       std_logic_vector(7 DOWNTO 0);
BEGIN
    bin_result<=astring+bstring;
    PROCESS
    BEGIN
        IF bin_result>"01001"
            THEN bcd_result<=bin_result+"0110";
            ELSE bcd_result<=bin_result;
        END IF;
    END PROCESS;
  END arc;
```

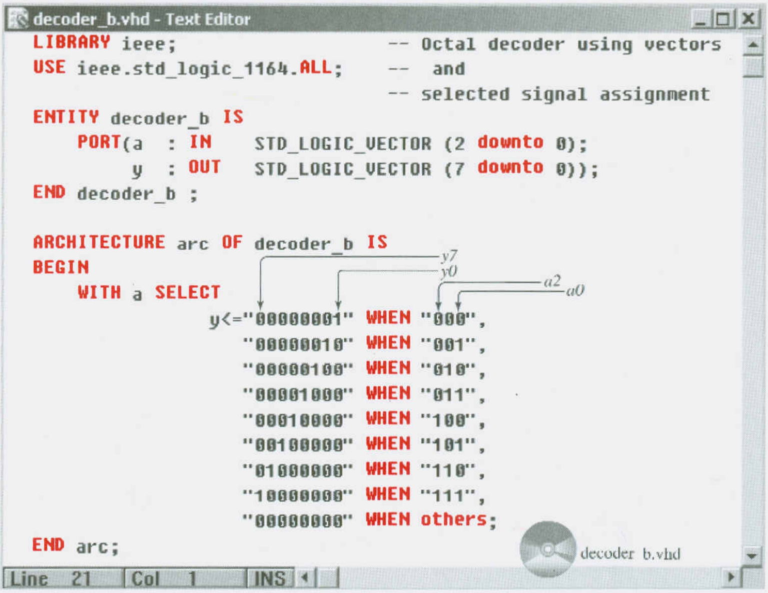
Internal interconnection signal

IF-THEN-ELSE sequential statements

④

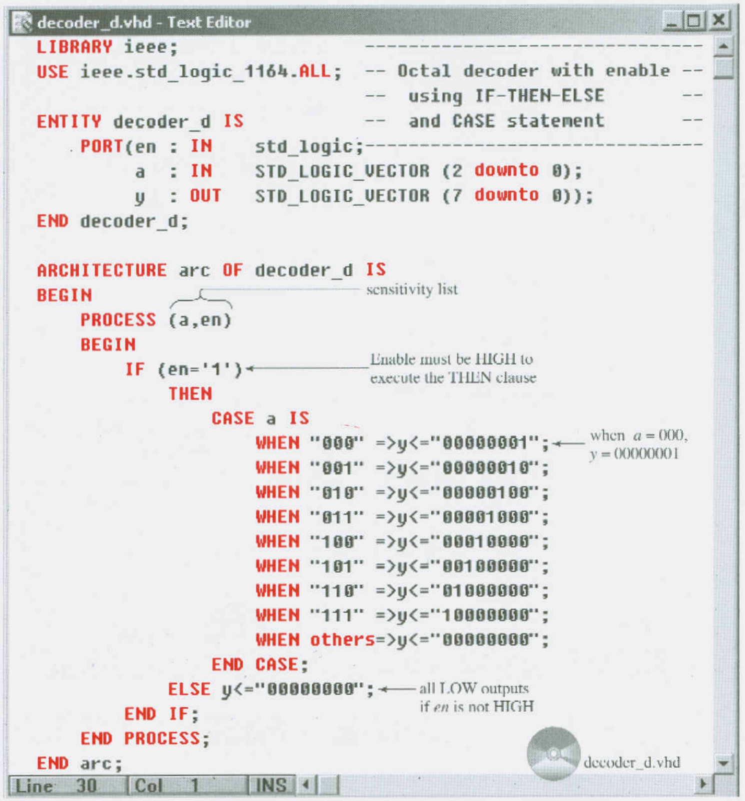

compare_8b.vhd - Text Editor
Sensitivity list
alb
aeb
agb
3-bit internal SIGNAL result
Assign vector elements to outputs
compare_8b.vhd
Line 27 Col 1 INS

```
LIBRARY ieee;                           ----------------------------
USE ieee.std_logic_1164.ALL;            -- 8-bit Comparator using  --
                                        -- IF-THEN-ELSE and ELSIF  --
ENTITY compare_8b IS                    ----------------------------
    PORT(a, b                   : IN    std_logic_vector(7 DOWNTO 0);
         agb, aeb, alb          : OUT   std_logic);
END compare_8b;

ARCHITECTURE arc OF compare_8b IS
    SIGNAL result               : std_logic_vector(2 DOWNTO 0);
BEGIN
    PROCESS (a,b)
    BEGIN
        IF     a<b THEN
                 result     <=  "001";
        ELSIF  a=b THEN
                 result     <=  "010";
        ELSIF  a>b THEN
                 result     <=  "100";
        ELSE
                 result     <=  "000";
        END IF;
        agb <=  result(2);
        aeb <=  result(1);
        alb <=  result(0);
    END PROCESS;
END arc;
```



```
LIBRARY ieee;                          -- Octal decoder using vectors
USE ieee.std_logic_1164.ALL;           --  and
                                       -- selected signal assignment
ENTITY decoder_b IS
    PORT(a  : IN    STD_LOGIC_VECTOR (2 downto 0);
         y  : OUT   STD_LOGIC_VECTOR (7 downto 0));
END decoder_b ;

ARCHITECTURE arc OF decoder_b IS
BEGIN
    WITH a SELECT
                  y<="00000001" WHEN "000",
                     "00000010" WHEN "001",
                     "00000100" WHEN "010",
                     "00001000" WHEN "011",
                     "00010000" WHEN "100",
                     "00100000" WHEN "101",
                     "01000000" WHEN "110",
                     "10000000" WHEN "111",
                     "00000000" WHEN others;
END arc;
```

⑤



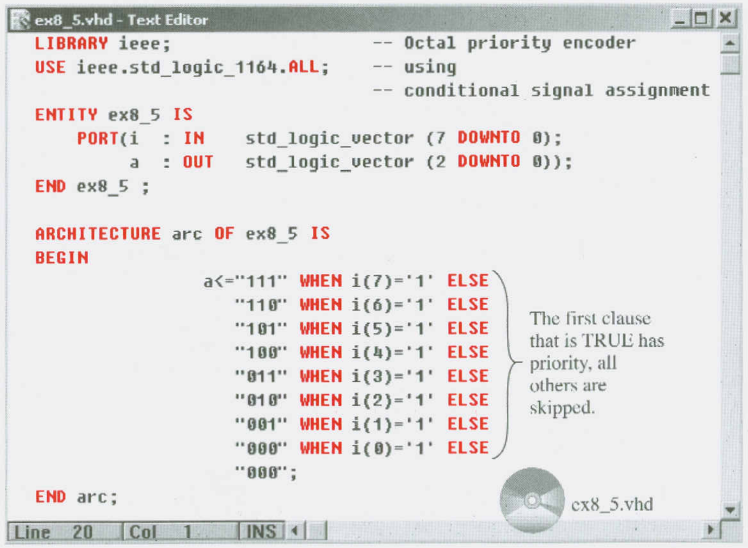
```
LIBRARY ieee;                          ----------------------------------
USE ieee.std_logic_1164.ALL;           -- Octal decoder with enable --
                                       --   using IF-THEN-ELSE        --
ENTITY decoder_d IS                    --   and CASE statement        --
   PORT(en : IN     std_logic;----------------------------------
         a  : IN     STD_LOGIC_VECTOR (2 downto 0);
         y  : OUT    STD_LOGIC_VECTOR (7 downto 0));
END decoder_d;

ARCHITECTURE arc OF decoder_d IS
BEGIN
   PROCESS (a,en)
   BEGIN
      IF (en='1')
         THEN
            CASE a IS
               WHEN "000" =>y<="00000001";
               WHEN "001" =>y<="00000010";
               WHEN "010" =>y<="00000100";
               WHEN "011" =>y<="00001000";
               WHEN "100" =>y<="00010000";
               WHEN "101" =>y<="00100000";
               WHEN "110" =>y<="01000000";
               WHEN "111" =>y<="10000000";
               WHEN others=>y<="00000000";
            END CASE;
         ELSE y<="00000000";
      END IF;
   END PROCESS;
END arc;
```

sensitivity list

Enable must be HIGH to execute the THEN clause

when *a* = 000, y = 00000001

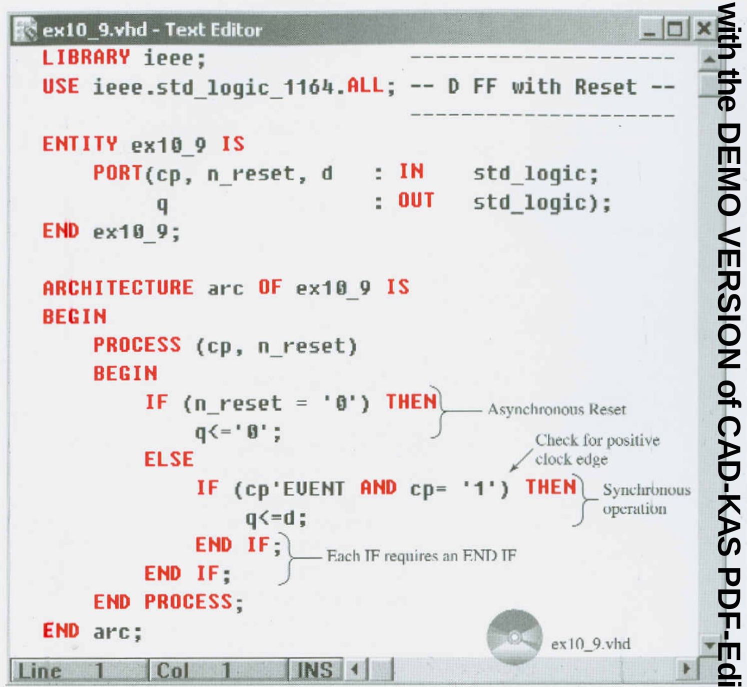
all LOW outputs if *en* is not HIGH



```
LIBRARY ieee;                    -- Octal priority encoder
USE ieee.std_logic_1164.ALL;     -- using
                                 -- conditional signal assignment
ENTITY ex8_5 IS
   PORT(i  : IN    std_logic_vector (7 DOWNTO 0);
        a  : OUT   std_logic_vector (2 DOWNTO 0));
END ex8_5 ;

ARCHITECTURE arc OF ex8_5 IS
BEGIN
               a<="111" WHEN i(7)='1' ELSE
                  "110" WHEN i(6)='1' ELSE
                  "101" WHEN i(5)='1' ELSE
                  "100" WHEN i(4)='1' ELSE
                  "011" WHEN i(3)='1' ELSE
                  "010" WHEN i(2)='1' ELSE
                  "001" WHEN i(1)='1' ELSE
                  "000" WHEN i(0)='1' ELSE
                  "000";

END arc;
```

The first clause that is TRUE has priority, all others are skipped.

⑥



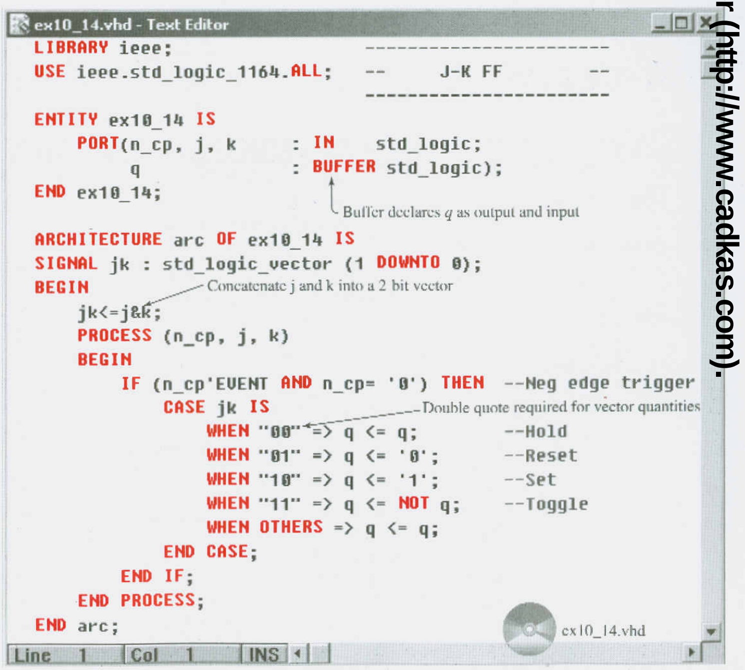
```
LIBRARY ieee;                    ------------------------
USE ieee.std_logic_1164.ALL; -- D FF with Reset --
                                 ------------------------
ENTITY ex10_9 IS
   PORT(cp, n_reset, d    : IN    std_logic;
        q                 : OUT   std_logic);
END ex10_9;

ARCHITECTURE arc OF ex10_9 IS
BEGIN
   PROCESS (cp, n_reset)
   BEGIN
      IF (n_reset = '0') THEN
         q<='0';
      ELSE
         IF (cp'EVENT AND cp= '1') THEN
            q<=d;
         END IF;
      END IF;
   END PROCESS;
END arc;
```

Asynchronous Reset

Check for positive clock edge

Synchronous operation

Each IF requires an END IF



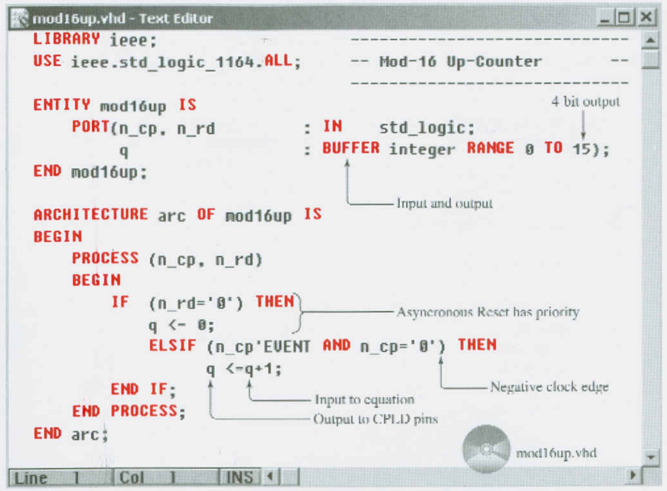
```
LIBRARY ieee;                    ------------------------
USE ieee.std_logic_1164.ALL;     --       J-K FF       --
                                 ------------------------
ENTITY ex10_14 IS
   PORT(n_cp, j, k      : IN     std_logic;
        q               : BUFFER std_logic);
END ex10_14;

ARCHITECTURE arc OF ex10_14 IS
SIGNAL jk : std_logic_vector (1 DOWNTO 0);
BEGIN
   jk<=j&k;
   PROCESS (n_cp, j, k)
   BEGIN
      IF (n_cp'EVENT AND n_cp= '0') THEN  --Neg edge trigger
         CASE jk IS
            WHEN "00" => q <= q;          --Hold
            WHEN "01" => q <= '0';        --Reset
            WHEN "10" => q <= '1';        --Set
            WHEN "11" => q <= NOT q;      --Toggle
            WHEN OTHERS => q <= q;
         END CASE;
      END IF;
   END PROCESS;
END arc;
```

Buffer declares *q* as output and input

Concatenate j and k into a 2 bit vector

Double quote required for vector quantities

⑦



```
LIBRARY ieee;                          -------------------------------
USE ieee.std_logic_1164.ALL;           -- Mod-16 Up-Counter          --
                                       -------------------------------
ENTITY mod16up IS
   PORT(n_cp, n_rd            : IN     std_logic;
        q                     : BUFFER integer RANGE 0 TO 15);
END mod16up:

ARCHITECTURE arc OF mod16up IS
BEGIN
   PROCESS (n_cp, n_rd)
   BEGIN
      IF  (n_rd='0') THEN
          q <- 0;
          ELSIF (n_cp'EVENT AND n_cp='0') THEN
                q <-q+1;
      END IF;
   END PROCESS;
END arc;
```

4 bit output

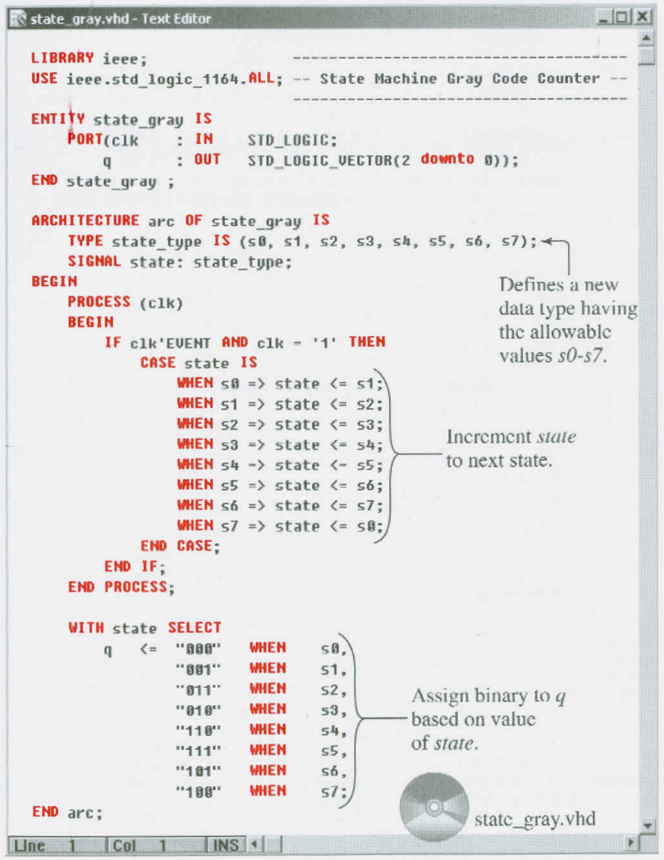
Input and output

Asyncronous Reset has priority

Negative clock edge

Input to equation

Output to CPLD pins



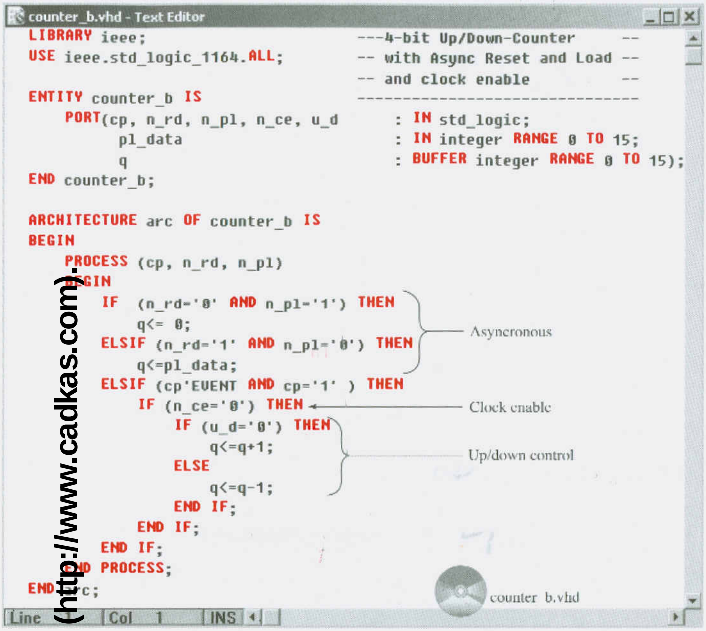
```
LIBRARY ieee;                  -------------------------------------
USE ieee.std_logic_1164.ALL; -- State Machine Gray Code Counter --
                               -------------------------------------
ENTITY state_gray IS
   PORT(clk     : IN    STD_LOGIC;
        q       : OUT   STD_LOGIC_VECTOR(2 downto 0));
END state_gray ;

ARCHITECTURE arc OF state_gray IS
   TYPE state_type IS (s0, s1, s2, s3, s4, s5, s6, s7);
   SIGNAL state: state_type;
BEGIN
   PROCESS (clk)
   BEGIN
      IF clk'EVENT AND clk - '1' THEN
         CASE state IS
            WHEN s0 => state <= s1;
            WHEN s1 => state <= s2;
            WHEN s2 => state <= s3;
            WHEN s3 => state <= s4;
            WHEN s4 -> state <- s5;
            WHEN s5 => state <= s6;
            WHEN s6 => state <= s7;
            WHEN s7 => state <= s0;
         END CASE;
      END IF;
   END PROCESS;

   WITH state SELECT
      q   <=  "000"   WHEN   s0,
              "001"   WHEN   s1,
              "011"   WHEN   s2,
              "010"   WHEN   s3,
              "110"   WHEN   s4,
              "111"   WHEN   s5,
              "101"   WHEN   s6,
              "100"   WHEN   s7;
END arc;
```

Defines a new data type having the allowable values *s0-s7*.

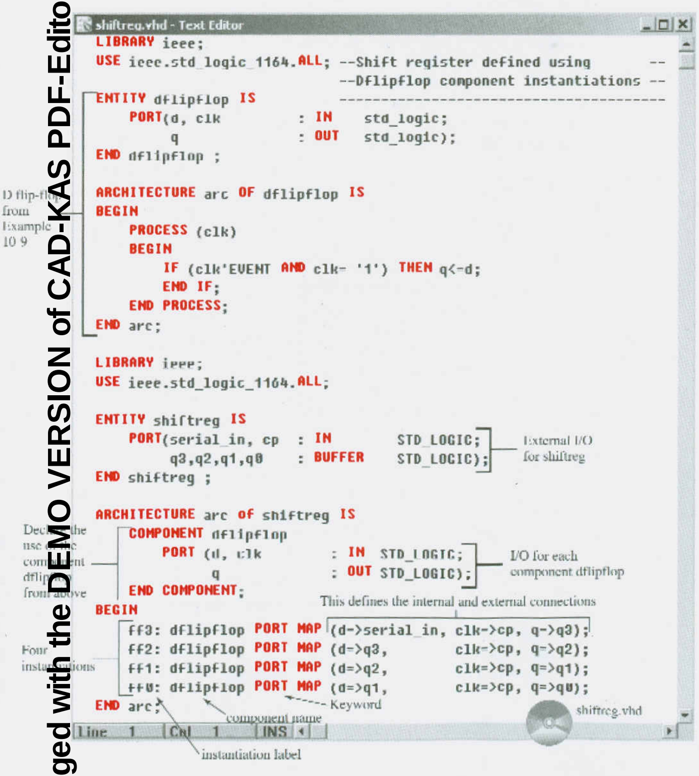
Increment *state* to next state.

Assign binary to *q* based on value of *state*.

©2004 by Pearson Educatio



```
LIBRARY ieee;                              ---4-bit Up/Down-Counter     --
USE ieee.std_logic_1164.ALL;               -- with Async Reset and Load --
                                           -- and clock enable          --
ENTITY counter_b IS                        --------------------------------
    PORT(cp, n_rd, n_pl, n_ce, u_d       : IN std_logic;
         pl_data                         : IN integer RANGE 0 TO 15;
         q                               : BUFFER integer RANGE 0 TO 15);
END counter_b;

ARCHITECTURE arc OF counter_b IS
BEGIN
    PROCESS (cp, n_rd, n_pl)
    BEGIN
        IF  (n_rd='0' AND n_pl='1') THEN
            q<= 0;
        ELSIF (n_rd='1' AND n_pl='0') THEN
            q<=pl_data;
        ELSIF (cp'EVENT AND cp='1' ) THEN
            IF (n_ce='0') THEN
                IF (u_d='0') THEN
                    q<=q+1;
                ELSE
                    q<=q-1;
                END IF;
            END IF;
        END IF;
    END PROCESS;
END arc;
```


shiftreg.vhd - Text Editor
D flip-flop from Example 10 9
External I/O for shiftreg
Declare the use of the component dflipflop from above
I/O for each component dflipflop
This defines the internal and external connections
Four instantiations
Keyword
component name
instantiation label
shiftreg.vhd
Line 1
Col 1
INS

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL; --Shift register defined using       --
                             --Dflipflop component instantiations --
                             --------------------------------------
ENTITY dflipflop IS
    PORT(d, clk          : IN    std_logic;
         q               : OUT   std_logic);
END dflipflop ;

ARCHITECTURE arc OF dflipflop IS
BEGIN
    PROCESS (clk)
    BEGIN
        IF (clk'EVENT AND clk= '1') THEN q<=d;
        END IF;
    END PROCESS;
END arc;

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY shiftreg IS
    PORT(serial_in, cp  : IN       STD_LOGIC;
         q3,q2,q1,q0    : BUFFER   STD_LOGIC);
END shiftreg ;

ARCHITECTURE arc of shiftreg IS
    COMPONENT dflipflop
        PORT (d, clk             : IN  STD_LOGIC;
              q                  : OUT STD_LOGIC);
    END COMPONENT;
BEGIN
    ff3: dflipflop PORT MAP (d->serial_in, clk->cp, q->q3);
    ff2: dflipflop PORT MAP (d=>q3,        clk->cp, q=>q2);
    ff1: dflipflop PORT MAP (d=>q2,        clk=>cp, q=>q1);
    ff0: dflipflop PORT MAP (d=>q1,        clk=>cp, q=>q0);
END arc;
```